

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas
TA: Mariana Capuñay

13 de septiembre de 2025

U2.1 Velocidad, eficiencia, escalabilidad. Ley de Amdahl/Gustafson

Dado el **speedup** (velocidad):

$$S = \frac{T_s}{T_p}$$

En general, $S \leq p$ La **eficiencia** es la **carga promedio** por procesador:

$$E(n) = \frac{S}{p} = \frac{T_s}{p \cdot T_p}$$

En general, si $S \leq p$, $E \leq 1$.

Si $S=p$, $E=1$

Ley de Amdahl - strong scaling

Responde la pregunta sobre que tan rápido ejecuta un código en paralelo cuando utilizo p procesos. También llamado **strong scaling**, donde el tamaño del problema es constante (condición rígida, fuerte)

El **tiempo de ejecución secuencial** (normalizado a la unidad) es:

$T_s = f_s + f_p$, con la fracción de código secuencial f_s , y paralela f_p ($f_s + f_p = 1$)

Resolverlo en p procesadores, implica para el **tiempo de ejecución en paralelo**:

$$T_p = f_s + \frac{f_p}{p}$$

$$S = \frac{f_s + f_p}{f_s + f_p/p} = \frac{1}{f_s + f_p/p}$$

$$E = \frac{1}{p(f_s + \frac{1-f_s}{p})}$$

Para $p \rightarrow \infty$, $S = 1/f_s$ (limitado por la parte secuencial)

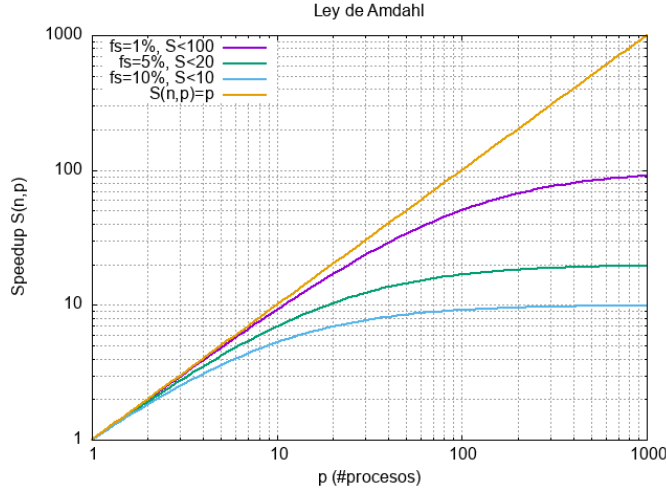


Figura 1: Ley de Amdahl

Ley de Gustafson - weak scaling

Dada una solución paralela, responde la pregunta sobre que más rápido se ejecuta en comparación a la versión secuencial, cuando el problema es de tamaño variable (creciente), en p procesos.

$$T_p = f_s + f_p$$

$$T_s = f_s + pf_p$$

$$S = \frac{f_s + pf_p}{f_s + f_p} = f_s + pf_p$$

$$E = \frac{f_s + pf_p}{p}$$

Speedup escala con el número de procesos p

Algunos autores derivan la ley de Gustafson bajo un supuesto distinto. Siendo el tiempo de ejecución para un problema de tamaño n variable ($n \propto p$):

$$T_s = f_s + f_p p^\alpha$$

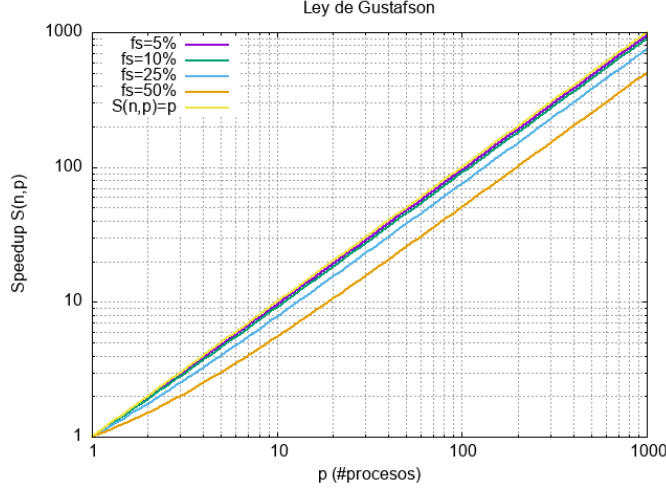


Figura 2: Ley de Gustafson

$$T_p = f_s + f_p p^{\alpha-1}$$

α es un entero arbitrario mayor o igual a cero.

$$S = \frac{T_s}{T_p} = \frac{f_s + f_p p^\alpha}{f_s + f_p p^{\alpha-1}}$$

$$E = \frac{S}{p} = \frac{f_s + (1-f_s)p^\alpha}{(f_s + (1-f_s)p^{\alpha-1}) \cdot p} = \frac{f_s p^{-\alpha} + (1-f_s)}{f_s p^{1-\alpha} + (1-f_s)}$$

Si el trabajo es $f_s + f_p p^\alpha$,

Para $\alpha = 0$, $E = \frac{1}{f_s p + (1-f_s)}$, deriva en **strong scaling** (Amdahl).

Para $\alpha = 1$, $E = f_s p^{-1} + (1-f_s)$

Asimismo, este problema se postula en función al **performance** (P= trabajo/tiempo).

Siendo:

$$P_s = \frac{f_s + f_p p^\alpha}{T_s} = 1$$

y en paralelo:

$$P_p = \frac{f_s + f_p p^\alpha}{T_p} = \frac{f_s + f_p p^\alpha}{f_s + f_p p^{\alpha-1}}$$

$$S = \frac{P_p}{P_s}$$

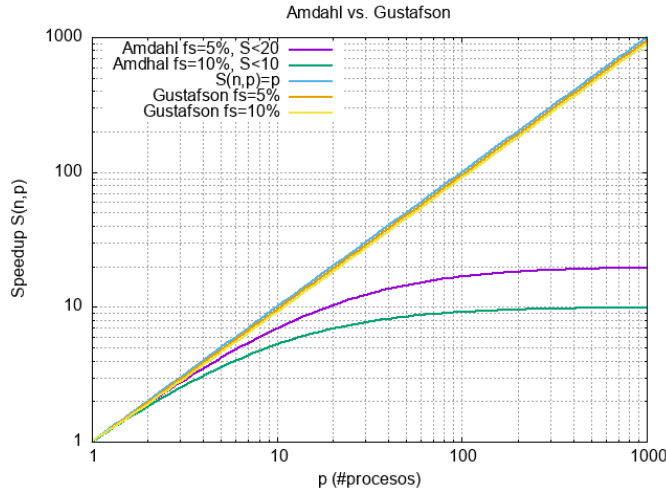


Figura 3: Ley de Amdahl vs. Gustafson

Escalabilidad

La escalabilidad evalúa el performance proporcional a un creciente número de procesos.

La escalabilidad mantiene una **eficiencia constante** si se incrementa el número de procesos (p) a la vez que la carga (n).

Esta propiedad no puede describirse con la Ley de Amdahl.

U2.2 Speedup, FLOPs

IEEE754 especifica un número de coma flotante (FLOP) de precisión simple de 32-bit base-2.

Speedup es la velocidad máxima de ejecución del total de operaciones de coma flotante por unidad de tiempo.

Se mide en:

$$FLOP/sec \quad (FLOPs) \quad (1)$$

Microprocesadores que generan dos o cuatro double precision por clock cycle con frecuencias entre 2-3 GHz obtienen:

$$4 - 12 \text{ GFLOPs por core} \quad (2)$$

Velocidad de memoria:

$$GB/sec \quad (3)$$

Cómputo:

$$GFLOP/sec \quad (4)$$

Eliminación de Gauss

Un sistema $Ax = b$ con n ecuaciones necesita aproximadamente:

$$n^3 FLOP \quad (5)$$

Tiempo de cómputo (1 GFLOPS):

n	FLOP	tiempo(sec)
10	10^3	10^{-6}
100	10^6	0.001
1000	10^9	1
10000	10^{12}	1000
100000	10^{15}	10^6
1000000	10^{18}	32 años

Además:

$$10^6 \times 10^6 = 10^{12} \text{ elementos} \quad (6)$$

Ocupa aproximadamente:

$$8 TB \quad (7)$$

Referencias

[1] Peter S. Pacheco. An Introduction to Parallel Programming. [2] Michael J. Quinn. Parallel Programming in C with MPI and OpenMP.